

Étude comparatives de méthodes de classification ECG

10 Juillet 2026

Contents

Abstract	1
Introduction	1
1 Signaux Électrocardiogrammes	1
2 Convolutional Dictionary Learning	2
2.1 Théorie	2
2.2 Application aux signaux ECG	3
3 Classifications	6
3.1 Définition des Classifieurs	6
3.2 Évaluation des classifieurs	6
3.3 Premières applications	7
3.4 Détermination de pics et extraction de caractéristiques	7
3.5 Comparaison des performances	9
3.6 Transformée en ondelettes et indicateurs spéciaux .	11
3.7 Résumé des performances du RandomForest	12
Conclusion	14
Reference	15
Annexe	15

Étudiant :
FAYADAS Hugo

Formation :
LDD3 MPSI

Encadrant :
Matthieu Kowalski

Abstract

The purpose of this paper is to display and compare classification processes applied to electrocardiogram (ECG) signals. With a main focus on the convolutional dictionary learning (CDL) process. We propose to apply a list of classical classifiers on the results of this process. We compare the estimation errors made with models trained on raw signals, reconstruct signals through CDL and atoms activation times. Also we evaluate the performance of models based on features of the signal, wavelet transform and interest ourselves in the influence of data conditioning on the results. We provide a complete and executable Python code for the reader's benefit (see Annexe).

Introduction

L'électrocardiographie est un examen médical visant à mesurer l'activité électrique du cœur, résultant en un tracé, l'électrocardiogramme (on fera l'amalgame entre les 2 par la suite). Il est principalement utilisé pour la détection et le diagnostic de nombreuses pathologies telles que les infarctus, l'arythmie, la fibrillation...

Toutefois, la détection d'anomalies au sein d'électrocardiogramme (ECG) est complexe, nécessitant du temps et du personnel qualifiée, rendant les diagnostics pourtant essentiels difficiles d'accès. De nombreuses études ont été menées afin d'automatiser ce processus l'objectif de cette étude est de comparer des méthodes de détermination de pathologies autonomes. On se concentrera principalement sur une méthode reposant sur le Convolutional Dictionary Learning (CDL). On comparera par la suite les performances de différents classifieurs appliqués aux résultats obtenus par le CDL. Mais aussi appliqués aux ECGs bruts, à un ensemble de leurs caractéristiques, leurs maxima locaux et leurs transformés en ondelettes. On s'intéresse aussi à la l'influence du conditionnement des données sur les performances de classifications.

1 Signaux Électrocardiogrammes

Cette partie est consacrée à la définition de l'objet de cette étude ; les signaux électrocardiogrammes. Un électrocardiogramme est l'enregistrement par voie cutanée de l'activité électrique du cœur. En pratique, il se réalise en plaçant douze électrodes sur la peau, on obtient 12 dérivation qui permettent de reconstruire le signal spatialement. Dans cette étude on se contentera d'une seule dérivation, soit d'un ECG monocanal. Les contractions du cœur sont déclenchées par une onde de dépolarisation électrique traversant les différentes cavités. On peut alors décomposer un battement en une séquence d'ondes conventionnelles comme on peut voir en figure 9.

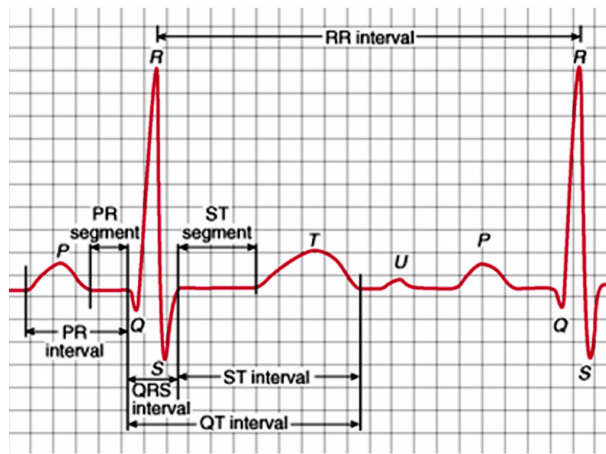


Figure 1: Schéma d'un battement de coeur classique.

On distingue l'onde P qui représente la contraction ou dépolarisation des oreillettes, L'onde

QRS, la contraction des deux ventricules, c'est le pic le plus net, le plus haut et l'élément le plus facilement détectable. Enfin, l'onde T est la relaxation ou repolarisation des ventricules en vu d'un nouveau battement celle ci est parfois suivi par l'onde U plus petite est moins longue que toutes les autres et à l'origine encore débattue voir [4].

Pour cette étude on dispose d'une base de données d'ECG monocanals contenant 162 ECGs triés par des professionnels en accès libre sur Physionet et téléchargeable [ici](#). Chacun d'eux échantillonné à une fréquence de 128Hz et d'une durée de 512s On dispose de trois types de signaux : les signaux de types NSR pour "Normal Sinus Ryhtm" soit un ECG sain, les signaux de types ARR pour "Arrythmia" et CHF pour "Congestive Heart Failure" (insuffisance cardiaque congestive). On représente les trois types de signaux en figures 2. On dispose de 96 signaux de type ARR, 36 de types NSR et 30 de type CHF.

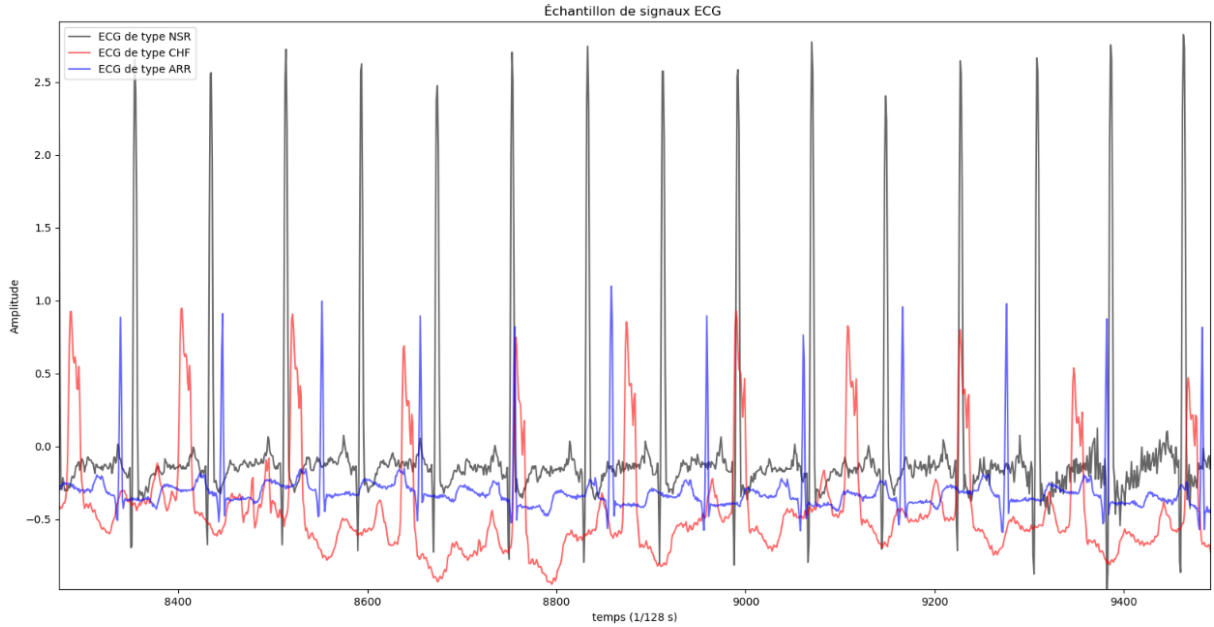


Figure 2: Extrait de signaux ECG de chacun des types

On observe sur cette figure des différences marquées au niveau des signaux mais surtout que pour chacun d'entre eux un même motif semble se répéter.

2 Convolutional Dictionary Learning

Dans cette partie on s'intéresse à l'algorithme de Convolutional Dictionary Learning (CDL). Dans un premier on définira le concept de CDL avant d'en faire l'utilisation sur les données ECGs présentées auparavant.

2.1 Théorie

L'objectif du Convolutional Dictionary Learning est de trouver une représentation parcimonieuse des données. Soit une représentation composée principalement de zéro permettant de compresser le signal et de mettre en valeur les éléments les plus significatifs. Pour cela on souhaite représenter chaque signal sous forme de combinaisons d'éléments de base communs à tous les signaux d'une même catégories. Ces éléments sont appelées atomes et ils forment un dictionnaire. Le but est de simultanément minimiser l'écart des données à la représentation et de la rendre la plus parcimonieuse possible. On peut formaliser ces concepts ainsi : soit X les signaux d'un certain type où $X \in \mathbb{R}^{n \text{ trials} \times n \text{ times}}$ où $n \text{ trials}$ et $n \text{ times}$ représentent respectivement le nombre de signaux et leur longueur. Soit D le dictionnaire dont les colonnes sont les atomes, c'est la matrice des coefficients de la combinaison qui permet de retrouver X . Avec $D \in \mathbb{R}^{n \text{ atoms} \times n \text{ times atoms}}$ où

n atoms est le nombre d'atomes, et n times atoms leur longueur ce sont des paramètres à fixer. Enfin Z est le tenseur de représentation parcimonieuse, avec $Z \in \mathbb{R}^{n \text{ trials} \times n \text{ atoms} \times \text{times atoms}}$. Le problème d'optimisation s'écrit alors :

$$\min_{D,Z} \frac{1}{2} \sum_{i=1}^{n \text{ trials}} \left\| X_i - \sum_{k=1}^{n \text{ atoms}} d_k * z_{i,k} \right\|_2^2 + \lambda \sum_{i=1}^{n \text{ trials}} \sum_{k=1}^{n \text{ atoms}} \|z_{i,k}\|_1$$

sous la contrainte : $\|d_k\|_2^2 \leq 1 \quad \forall k \in \{1, \dots, n \text{ atoms}\}$

Le premier terme de cette formule a pour objectif d'assurer la correspondance entre la représentation et les données. La convolution permet de répliquer les atomes le long du signal tandis que les $z_{i,k}$ propres à chaque signal indiquent quand et avec quelles amplitudes les répliquer, ce sont des "cartes d'activation" ou signaux d'activation. Le second terme quant à lui sert à imposer le caractère parcimonieux à ces cartes (qu'on appelle aussi temps d'activation). Le paramètre lambda permettant de contrôler l'importance que l'on donne à ce caractère, pour un lambda très élevé le modèle cherchera à mettre des zéros partout quitte à n'avoir plus qu'une ligne droite, à l'inverse un lambda trop faible conduirait à avoir des atomes s'activant en permanence, on perdrait les avantages de cette représentation. On donne un exemple schématique de ce processus en figure 3.

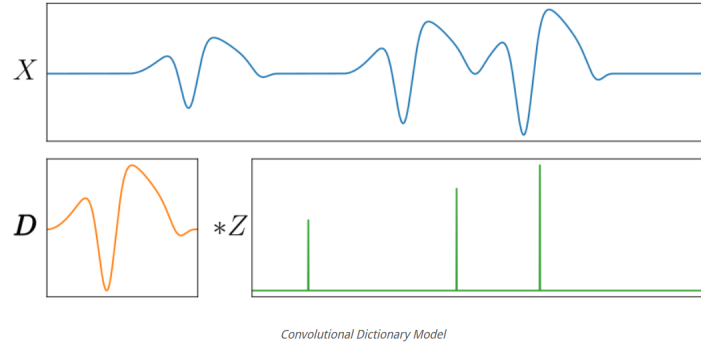


Figure 3: Décomposition d'un signal unidimensionnel sans bruit (bleu) sous forme de convolution entre un motif temporel, atome (orange) et un signal d'activation parcimonieux (vert).

Pour optimiser ce problème on fixe tour à tour D , et Z et on estime l'autre avec une descente de gradient pour le terme quadratique et un algorithme plus complexe pour le second faisant intervenir notamment une fonction de seuil¹. Pour plus de précisions voir [8] et [3]. Pour reconstruire un signal il suffit alors de réaliser le produit de convolution entre les atomes de sa catégorie et ses temps d'activations propres. Soit

$$X_i = \sum_{k=1}^{n \text{ atoms}} z_{i,k} * d_k$$

2.2 Application aux signaux ECG

En pratique, on utilise le module `alphasc` de python^{[8],[2]} qui contient la fonction "learn_d_z" qui résout le problème d'optimisation précédent en fonction de paramètres à fixer. Ces paramètres sont : le nombre d'itérations de l'algorithme, le nombre d'atomes à rechercher, leur longueur et le coefficient de parcimonie λ . Pour fixer ces paramètres on s'inspire des signaux bruts et des motifs qu'on y décèle (voir figure 2) et on procède par tâtonnements jusqu'à obtenir un résultat cohérent. Surtout pour la reconstruction des signaux.

¹Cette algorithme n'étant pas le sujet de cette étude et des fonctions correspondantes étant déjà à disposition on ne s'attardera pas plus sur celui-ci

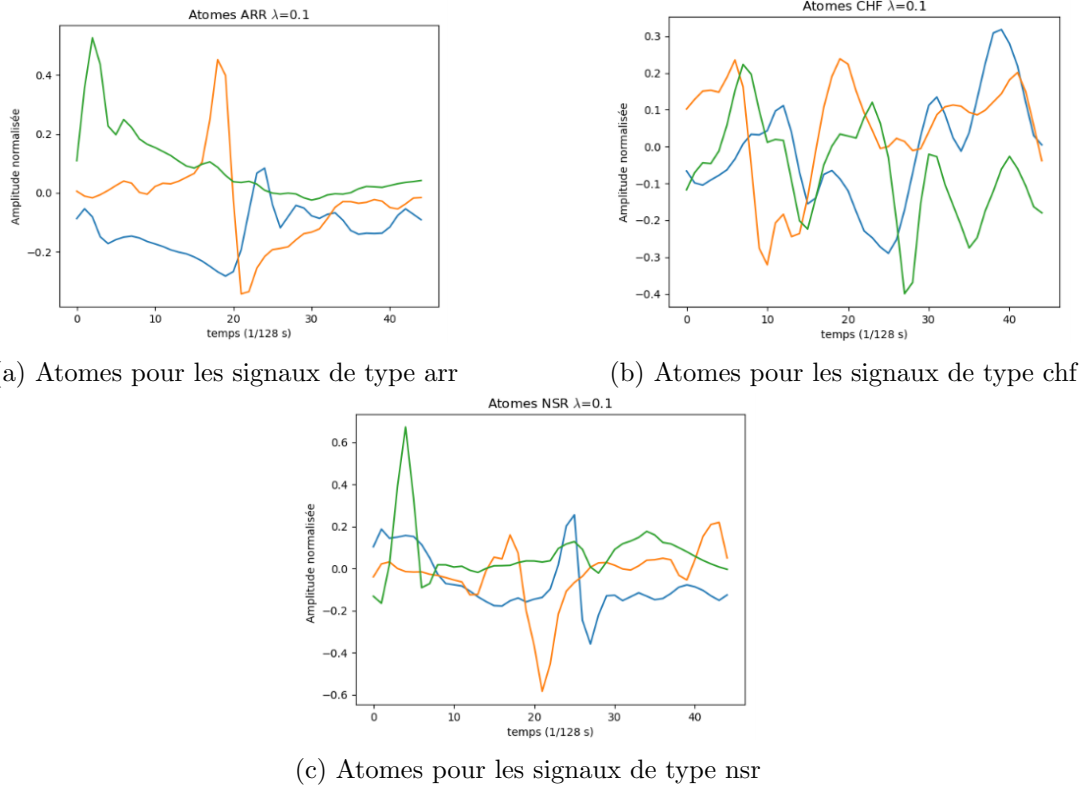


Figure 4: Atomes obtenus pour $\lambda = 0.1$

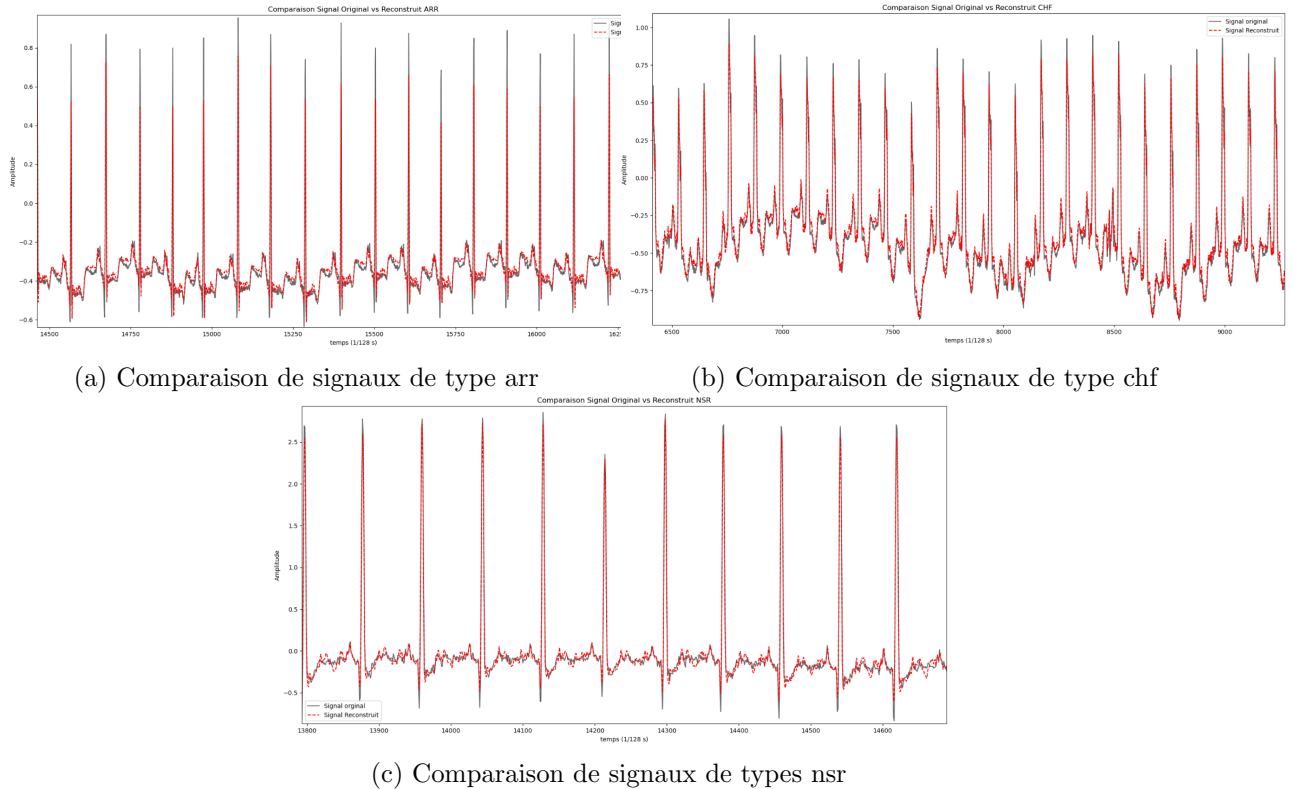


Figure 5: Comparaison de signaux bruts et reconstruits avec $\lambda = 0.1$

On présente ici les meilleurs résultats obtenus qui correspondent à 50 itérations et 3 atomes de longueurs 45 mesures (soit $45/128=0.35s$). On distingue deux coefficients de parcimonie λ afin de montrer l'impact de la parcimonie sur les résultats. En figure 4 on présente les atomes obtenus dans le cas où $\lambda = 0.1$ tandis que dans la figure 6 $\lambda = 1$ on peut déjà constater des

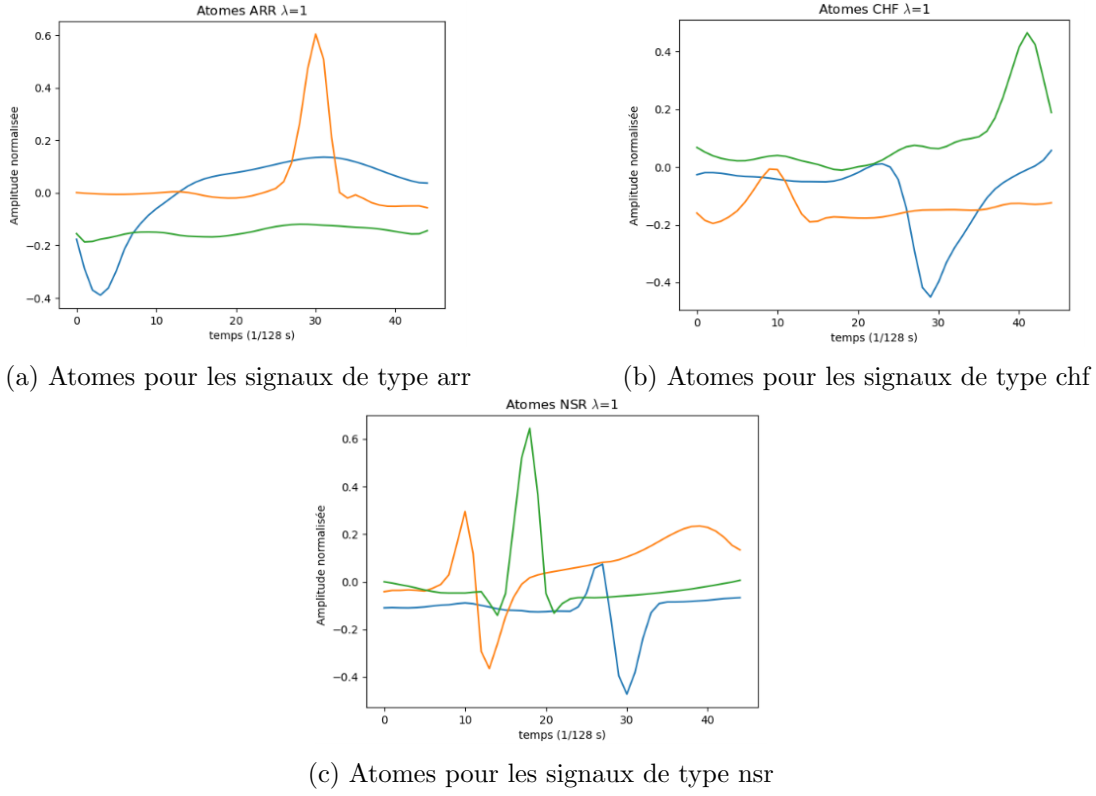


Figure 6: Atomes obtenus pour $\lambda = 1$

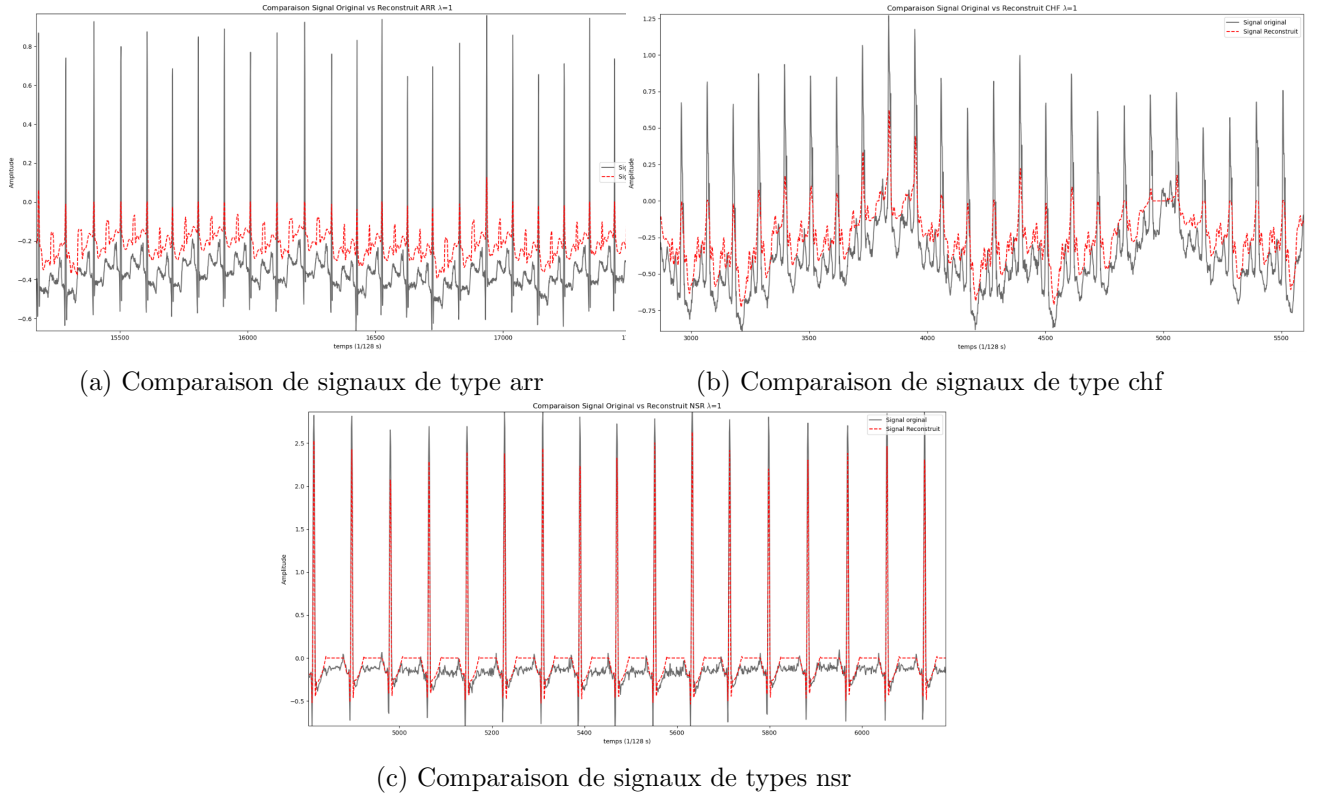


Figure 7: Comparaison de signaux bruts et reconstruits avec $\lambda = 1$

différences au niveau de la formes des atomes qui sont biens plus détaillés dans le premier cas ce qui est rendu possible par des signaux d'activations moins parcimonieux. On représente dans les figures 5 et 7 des échantillons de signaux reconstruit de tous types avec respectivement $\lambda = 0.1$ et $\lambda = 1$ on peut une nouvelle fois constater que dans le premier cas l'approximation est bien

plus proche des données originelles

3 Classifications

L'objectif de cette partie est de comparer les performances de classifieurs quant à la détermination des types des signaux ECGs. Pour cela on donne ici une brève définition des classifieurs. On expose les modalités d'évaluation de leurs performances avant de les appliquer aux données établies à la partie précédente. On définit ensuite des méthodes pour former de nouveaux éléments à partir des signaux bruts. On détermine les temps correspondants aux pics des signaux en premier lieu puis on extrait un ensemble de caractéristiques. On applique les classifieurs sur ces nouveaux éléments et comparons les résultats ainsi acquis. Après quoi on utilise la transformée en ondelettes et des modèles d'auto-régression et on a un nouveau jeu de données sur lesquels on applique les classifieurs. Enfin, on présente un résumé et compare les performances obtenues avec le meilleur des classifieurs.

3.1 Définition des Classifieurs

En eux même les calculs exécutés précédemment ne permettent pas la détermination automatisée du type d'un signal. Pour cela on fait appel à des classifieurs. Un classifieur est un algorithme de prédiction que l'on a entraîné en lui présentant un grand nombre de données avec les étiquettes correspondantes et qui lorsqu'on lui présente une nouvelle donnée inconnue va pouvoir prédire l'étiquette correspondante en utilisant ce qu'il a appris. Cela rend les classifieurs spécifiquement intéressants pour notre étude, les classifieurs les plus couramment utilisés sont ceux du module scikit-learn de python^{[5],[1]} c'est ceux que l'on utilisera par la suite. Les classifieurs que nous utiliserons sont les suivants : LogisticRegression, DecisionTreeClassifier, RandomForest, GaussianNB, BernoulliNB, KNeighbors. Il est à noter que certains sont initialement définis comme des classifieurs binaires (ne pouvant que séparer 2 catégories) on décide toutefois de les conserver pour la suite afin d'observer l'impact que cela a sur leurs performances. Sachant qu'ils ont naturellement tendance à procéder en opposant toutes les catégories contre une successivement pour chacune des catégories.

Compte-tenu des performances respectives des classifieurs au cours des tests on présente ici brièvement le fonctionnement théorique d'un des classifieurs qui se sont révélés les plus performants. Le classifieur RandomForest repose sur le principe des arbres de décisions, on avance dans l'arbre en répondant à des questions basiques à chaque palier. Par exemple, si on voulait faire un arbre de décision sur les moyens de transports une des premières questions pourrait être le véhicule possède-t-il des roues si oui, une seconde question : "plus de deux roues ?" si non c'est sûrement une moto et ainsi de suite. Toutefois les arbres de décisions sont souvent confrontés au problème d'overfitting, ils apprennent parfaitement les données mais sont trop complexes et détaillés pour être utiles sur de nouvelles données. Pour cela au lieu de faire grandir un seul arbre le RandomForest fait croître un grand nombre d'arbres moyens qu'ils n'entraînent pas exactement de la même façon. Ils n'ont qu'à une portion aléatoire de celles-ci, de plus ils ne peuvent se baser que sur un ensemble aléatoire de questions à chaque tour et non plus sur l'ensemble. Ainsi, tous les arbres formés ne donnent pas la même réponse et au moment de classer une nouvelle donnée l'étiquette qui a le plus de vote parmi les arbres l'emporte. Ce mode de fonctionnement fait du RandomForest l'un des classifieurs les plus robustes existants.

3.2 Évaluation des classifieurs

Afin d'évaluer les performances des classifieurs lors des tests suivants on va calculer des scores pour chacun d'entre eux. Ces scores sont le taux d'erreur, la précision, le rappel (ou recall) et le f1-score. La précision d'un modèle pour une catégorie est définie comme le rapport du nombre de fois où elle a été correctement prédite pour le jeu de données de test sur le nombre de fois où elle a été prédite sur ce même jeu. Le rappel correspond lui au rapport entre le nombre de fois où une classe a été correctement prédite au sein du jeu de test et le nombre de fois où cette classe

apparaît dans le jeu de test. Pour mieux comprendre la différence entre les deux, imaginons que notre modèle nous renvoie toujours la même classe alors le rappel de cette classe serait de 100%, toutes les données appartenant à cette classe auraient été trouvées. La précision elle serait très mauvaise puisque l'on aurait associé à cette classe nombre de données qui n'ont rien à voir. Dans l'idéal donc à maximiser simultanément ces deux scores, c'est pour cela faire que l'on calcule le f1-score. C'est la moyenne harmonique des scores précédents. Enfin, le taux d'erreur (aussi noté err rate) est le nombre d'étiquettes mal attribuées sur la taille du test elle contient moins de précision que les autres mais permet des comparaisons rapides entre les classifieurs.

3.3 Premières applications

Dans un objectif de comparaison on va appliquer les classifieurs aux signaux bruts, à leurs temps d'activation et aux signaux reconstruits. Pour pouvoir appliquer les classifieurs à nos données il nous faut les diviser en données d'entraînement et de test. On sélectionne donc aléatoirement un pourcentage élevé, typiquement entre 70% et 90% de chacune des trois classes des signaux étudiés on les mélange aléatoirement et on obtient le jeu d'entraînement. On conserve évidemment l'étiquette correspondant à chacun des signaux. On utilise le reste des signaux que l'on dispose aléatoirement en tant que jeu de test. On applique alors le classifieur sur les données ainsi formées et on note les scores évoqués précédemment. On moyenne ces scores en réalisant ce processus complet cent fois par items. On décide de voir si en appairant deux catégories et en l'opposant à la troisième on obtient de meilleurs résultats. Pour cela on définit une variable de tri qui prend les valeurs : classic, chf, nsr1 et nsr2. Elles désignent respectivement le cas où on sépare les trois variables, le cas où on associe les signaux ARR et CHF ensemble (sous le label CHF), le cas où on associe les signaux ARR et NSR ensemble (sous le label NSR) et enfin le cas où on associe les signaux NSR et CHF ensemble (sous le label NSR). Dans la suite de cette étude on ne s'intéresse plus qu'aux résultats pour lesquels $\lambda = 0.1$, les résultats avec $\lambda = 1$ étant systématiquement plus mauvais. Dans les tableaux 1, 2 et 3 on présente les taux correspondant à chaque modèle et à chaque configuration. En premier, lieu on constate que les résultats donnés par les signaux bruts sont très mauvais comme on l'attendait puisque ils comportent trop de points pour que les classifieurs puissent être efficaces. On ne s'intéressera donc plus à cette configuration dans la suite de cette étude. Dans un second temps On constate que le classifieur RandomForest est souvent meilleur. Par ailleurs, on note qu'aucun des classifieurs pour les 2 jeux de données ne donnent de résultats satisfaisant pour les modes de tri 'classic', 'chf' et 'nsr2'. À l'inverse, lorsque l'on se base sur les temps d'activations on obtient de très bon résultat pour le mode de tri 'nsr1'. Ce qui laisse à penser que les signaux de types ARR et NSR ont des temps d'activations très similaires et que ceux ci ne recèlent donc pas la majorité de l'information. Au contraire des temps d'activations des signaux de type CHF qui semblent avoir des temps d'activations assez différents et porteurs d'une certaine part d'information. On peut confirmer cette impression en analysant les autres scores. En figure 8 on compare les scores obtenus pour le classifieur LogisticClassifier avec les temps d'activation pour les tri classic et nsr1. Au niveau du tri classic on peut voir de faibles scores pour les signaux de types NSR². Ce qui confirme l'idée que les temps d'activations ne comporte que peu d'information pour ce type de signal. Tandis que lors du passage au tri nsr1 on obtient de bien meilleures scores³, tendance qui se confirme dans les autres tables de score (cf Annexe).

3.4 Détermination de pics et extraction de caractéristiques

On décide de se servir des données brutes pour calculer de nouveaux éléments. Dans un premier on se concentre sur les pics QRS de chaque battement, on définit une fonction de seuil et on extrait les abscisses correspondant aux pics des signaux. On applique aux nouvelles données obtenues l'ensemble du processus de classification décrit précédemment. Dans un second on décide d'extraire des caractéristiques fondamentales de chacun des signaux.

²dans ces tableaux les types de signaux sont uniquement indiqués par leur première lettre

³On rappelle que dans le cadre du tri nsr1 les signaux de types ARR et NSR sont regroupés sous le label NSR

Classifieur	Taux d'erreur (Tri Classic)
Logistic Regression	0.444
Decision Tree	0.437
Random Forest	0.281
Gaussian NB	0.627
Bernoulli NB	0.488
KNN ($k = 10$)	0.379
KNN ($k = 11$)	0.381
KNN ($k = 12$)	0.378

Table 1: Comparaison des taux d'erreur avec les signaux bruts et le tri classic

Modèle (Activation)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.245	0.215	0.043	0.236
Decision Tree	0.344	0.181	0.163	0.312
Random Forest	0.173	0.128	0.079	0.245
Gaussian NB	0.411	0.238	0.175	0.290
Bernoulli NB	0.261	0.204	0.159	0.315
KNN ($k = 10$)	0.338	0.236	0.078	0.329
KNN ($k = 11$)	0.331	0.234	0.090	0.328
KNN ($k = 12$)	0.346	0.233	0.087	0.350

Table 2: Comparaison des taux d'erreur avec les temps d'activation.

Modèle (Reconstruct)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.407	0.234	0.164	0.445
Decision Tree	0.338	0.185	0.235	0.328
Random Forest	0.281	0.157	0.167	0.280
Gaussian NB	0.404	0.261	0.210	0.417
Bernoulli NB	0.559	0.332	0.434	0.556
KNN ($k = 10$)	0.262	0.165	0.162	0.261
KNN ($k = 11$)	0.285	0.141	0.172	0.258
KNN ($k = 12$)	0.296	0.194	0.159	0.273

Table 3: Comparaison des taux d'erreur sur les signaux reconstruits.

(a) Scores pour LogisticClassifier avec tri classique				(b) Scores pour LogisticClassifier avec tri nsr1			
	Precision	Recall	F1_Score		Precision	Recall	F1_Score
A	71.689944	97.284211	82.489666	C	93.926587	82.704762	86.864205
C	91.865079	86.695238	88.343717	N	96.352148	98.628612	97.434692
N	66.833333	13.053571	21.271212				

Figure 8: Extrait des tables de scores pour les temps d'activations $\lambda = 0.1$

Les caractéristiques extraites sont celles ci : l'énergie, la variance, les valeurs maximales et minimales, le taux de passage par zéro, les quantiles, les fréquences, la fréquence moyenne, la dynamique d'évolution via la dérivée, le skewness et le kurtosis. Le skewness et le kurtosis aussi appelés coefficients d'asymétrie et d'aplatissement mesurent respectivement la symétrie du signal par rapport à sa moyenne et la répartition des points dans la queue de la courbe par

rapport à une distribution normale. Encore une fois, on y applique le processus de classification complet.

Modèle (Peaks)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.447	0.254	0.249	0.367
Decision Tree	0.437	0.228	0.257	0.398
Random Forest	0.379	0.195	0.217	0.336
Gaussian NB	0.627	0.583	0.170	0.406
Bernoulli NB	0.488	0.424	0.189	0.371
KNN ($k = 10$)	0.379	0.217	0.199	0.325
KNN ($k = 11$)	0.381	0.218	0.190	0.322
KNN ($k = 12$)	0.378	0.225	0.195	0.335

Table 4: Comparaison des taux d’erreur avec l’analyse des pics.

Modèle (Features)	Classic	CHF	NSR1	NSR2
Logistic Regression	0.292	0.098	0.211	0.264
Decision Tree	0.162	0.045	0.132	0.158
Random Forest	0.159	0.045	0.118	0.144
Gaussian NB	0.555	0.380	0.217	0.456
Bernoulli NB	0.418	0.235	0.185	0.415
KNN ($k = 10$)	0.416	0.233	0.203	0.401
KNN ($k = 11$)	0.403	0.235	0.207	0.392
KNN ($k = 12$)	0.402	0.229	0.207	0.385

Table 5: Comparaison des taux d’erreur avec les features extraites.

3.5 Comparaison des performances

À partir du tableau 4 on peut déjà remarquer qu’aucun classifieur et aucune configuration de tri ne donne de bons résultats à partir des pics QRS ce qui en fait de mauvais indicateurs. A contrario, dans l’ensemble les modèles qui s’appuient sur les features donne de meilleur résultat (voir tableau 5. Surtout, les trois premiers classifieurs dans la configuration de tri chf. Ce qui signifierait que l’information pour les signaux de types NSR est contenu principalement dans au moins certaines des caractéristiques explorées. De plus, en recoupant les informations données par ces modèles sur les features en tri chf et les résultats obtenus par les temps d’activation pour le tri nsr1 on peut déterminer avec une bonne précision les trois catégories.

Type d’Entrée	Classic	CHF	NSR1	NSR2
Brut (Raw)	0.281	-	-	-
Activation Time	0.173	0.128	0.079	0.245
Peaks	0.379	0.195	0.217	0.336
Features	0.159	0.045	0.118	0.144
Reconstruct	0.281	0.157	0.167	0.280

Table 6: Évolution du taux d’erreur du Random Forest selon le type d’entrée et la méthode de tri.

On présente dans le tableau 7 les taux d’erreurs moyens de l’ensemble des classifieurs par type d’entrée et de tri. Clairement les modèles basées sur les temps d’activations et les features sont les plus performants. Les temps d’activations permettent de meilleures prédictions pour le

Type de Signal	Classic	CHF	NSR1	NSR2
Activation Time	0.319	0.211	0.109	0.301
Peaks	0.440	0.294	0.210	0.358
Features	0.326	0.188	0.186	0.323
Reconstruct	0.366	0.208	0.200	0.363

Table 7: Taux d’erreur moyen de l’ensemble des classifieurs par type d’entrée et de tri.

Classifieur	Classic	CHF	NSR1	NSR2
Logistic Regression	0.348	0.200	0.167	0.328
Decision Tree	0.320	0.160	0.197	0.299
Random Forest	0.248	0.131	0.145	0.251
Gaussian NB	0.499	0.366	0.193	0.392
Bernoulli NB	0.432	0.299	0.242	0.414
KNN ($k = 10$)	0.349	0.213	0.161	0.328
KNN ($k = 11$)	0.350	0.207	0.165	0.325
KNN ($k = 12$)	0.356	0.220	0.162	0.336

Table 8: Taux d’erreur moyen de chaque algorithme par type de tri, moyenné sur l’ensemble des formes d’entrées.

tri nsr1 tandis que les features donnent les meilleurs résultats pour le tri chf. Ce qui confirme nos hypothèses de répartition des informations précédentes. Pour déterminer avec certitude le classifieur le plus performant on réalise le tableau 8 où on représente le taux d’erreur moyen de chaque algorithme par type de tri, moyenné sur l’ensemble des formes d’entrées. Il devient alors clair que le RandomForest est le meilleur de tous ces classifieurs pour notre problème. On consacre donc le tableau 6 à présenter l’évolution du taux d’erreur du RandomForest selon le type d’entrée et la méthode de tri. On y retrouve évidemment les excellentes performance pour les features en chf et les temps d’activations en nsr1. En outre, dans le tableau 8 on montre que les modèles Gaussian NB et Bernoulli NB soit les motifs naïfs de Bayes, sont généralement assez mauvais et on du mal à assimiler la complexité des données, leur approche probabiliste ne convient sans doute pas à nos données. Notamment au niveau des hypothèses réalisés par ces modèles. L’hypothèse d’indépendance (la naïveté) suppose que les caractéristiques des signaux sont indépendantes les unes des autres ce qui est quasiment toujours faux dans les entrées que l’on utilise. D’autre part ces classifieurs font respectivement les hypothèses gaussienne et binomiale des données ce qui est encore faux. Enfin, si on s’intéresse au KNeighbors ou k-nearest neighbors, on voit que le choix de k (nombre de voisins à considérer) n’influencent pas significativement les résultats.

Représentation du Signal	Temps Moyen d’Exécution
Signaux Bruts (<i>Raw</i>)	~ 466,5 secondes
Activation Time	~ 914,9 secondes
Peaks	~ 632,8 secondes
Features	~ 42,1 secondes
Reconstruct	~ 385,4 secondes

Table 9: Impact du type d’entrée sur le temps moyen d’entraînement et d’évaluation par modèle (Tri Classic).

Un autre élément important à prendre en compte est le temps de calcul nécessaire pour

Type de représentation d'entrée	Temps d'exécution (Random Forest - Classic)
Signaux Bruts (<i>Raw</i>)	365,09 s ($\sim$ 6 min 05)
Activation time	717,23 s ($\sim$ 11 min 57)
Peaks	85,78 s ($\sim$ 1 min 25)
Features	5,61 s
Reconstruct	334,14 s ($\sim$ 5 min 34)

Table 10: Temps de calcul du Random Forest selon la forme du signal d'entrée (Tri Classic).

obtenir chacun de ces résultats. Le tableau 9 indique le temps de calcul⁴ moyenné sur les modèles de classifieurs par type d'entrée pour le tri classic du processus de classification dans son ensemble (les 100 itérations). On en conclut que les features offrent le meilleur rapport coût/performances. Les temps d'activations biens que plus précis en terme général sont aussi les plus coûteux en termes de temps d'exécution, près de 22x le temps d'exécution des features en moyenne. Il faut aussi rappeler que la détermination des atomes et des temps d'activations est déjà un processus assez chronophage. Dans le tableau 10 on se concentre sur les temps de calculs du classifieur RandomForest où on repère que les temps d'activations nécessitent 12 min de calculs tandis que les features seulement 5s. Toutefois, il faut relativiser ce constat, ici ce qui prend le plus de temps c'est l'entraînement des modèles et le fait qu'on itère le processus entier 100 fois. En pratique on entraîne une unique fois le modèle et ensuite on l'utilise pour prédire des étiquettes une à une, il prédit la classe de l'ECG d'un patient.

3.6 Transformée en ondelettes et indicateurs spéciaux

Lorsqu'on prend les features des signaux on calcule entre autre leurs transformée de Fourier. Bien que la transformée soit utile elle présente un défaut majeur, elle donne les fréquences qui apparaissent dans le signal mais n'apporte aucune information quant au moment où elles apparaissent. Pour une musique par exemple on sait quels notes sont jouées mais pas dans quel ordre. On fait alors intervenir la transformée en ondelettes au lieu de servir d'ondes sinusoïdales infinies on utilise une fonction appelée ondelette mère localisée dans le temps qui commence et finit à zéro. On fait glisser cette fonction le long du signal quand les éléments se produisent. On comprime ou dilate cette fonction, si on comprime la fonction elle devient très courte et permet d'analyser soit les hautes fréquences. Si on la dilate elle s'allonge et permet d'analyser les basses fréquences du signal, c'est à dire les tendances globales.

En pratique on adapte le code matlab de [6] à notre problème en python et on s'inspire de [9]. Pour caractériser les signaux ECG en vue de leur classification, on adopte une approche combinant des descripteurs temporels, fréquentiels et multifractaux. L'extraction s'effectue selon deux échelles temporelles distinctes :

- Analyse locale (par blocs) : Chaque signal est segmenté en 8 blocs d'environ une minute. Sur chaque bloc, trois types de caractéristiques sont calculés : les coefficients d'un modèle autorégressif (AR) d'ordre 4 pour capturer la dynamique temporelle, l'entropie de Shannon issue d'une transformée en paquets d'ondelettes discrètes pour mesurer la régularité locale du signal.
- Analyse globale : Sur la totalité du signal, la variance des ondelettes multi-échelles est estimée de manière non biaisée. En utilisant l'ondelette 'db2' et en excluant les coefficients affectés par les conditions aux limites, cette variance est calculée sur un total de 14 niveaux de résolution.

⁴Il est important de noter que les calculs ont été effectués par un notebook python sur un unique cœur d'un cpu i5 de 12ème génération. Les performances pourrait donc être largement amélioré par un changement de matériel

Cette combinaison d'indicateurs permet d'obtenir une signature mathématique robuste et reconnue pour la discrimination des pathologies cardiaques. Pour plus de clarté on désigne désormais l'ensemble de ces indicateurs en tant que indicateurs spéciaux.

Un modèle auto régressif est un algorithme qui essaie de deviner la valeur du signal à un instant t en fonction des valeurs précédentes; On dit qu'il est d'ordre 4 si il utilise les 4 valeurs précédentes pour déterminer la valeur du signal à l'instant ce que l'on peut formaliser : $x_t \approx a_1x_{t-1} + a_2x_{t-2} + a_3x_{t-3} + a_4x_{t-4}$ où les a_i sont les coefficients du modèle soit ce que l'on conserve. Ils peuvent détecter les changements de rythme par exemple. On calcule l'entropie de Shannon sur les coefficients d'ondelettes. Une faible entropie indique que l'énergie est concentrée sur quelques coefficients d'ondelettes spécifiques ce qui indique que le cœur bat régulièrement. À l'inverse une entropie élevée montre une dispersion de l'énergie et un signal plus chaotique. L'ondelette dite 'db2' est la deuxième ondelette d'une famille très réputé : la famille de Daubechies. Elle est composée de pics asymétriques qui la rende particulièrement adapté à l'étude des signaux ECGs (voir la représentation des ondelettes de Daubechies en annexe). On calcule d'abord ses indicateurs sur les signaux. Puis, on y applique le classifieur RandomForest on obtient un taux d'erreur de 0.24 environ ainsi que le tableau 11. On obtient des résultats assez mauvais en procédant ainsi ce qu'on peut au moins en parti imputer à la conversion du code matlab en python. On décide de tester ce procédé sur les temps d'activations on obtient un taux d'erreur de seulement 0.06 et on affiche le tableau des scores voir 12. Tout les scores sont élevés, on distingue très bien les 3 types de classes sans avoir à les appairer. On obtient même de meilleurs résultat que la source [6] qui avait un taux d'erreur de 0.8 et des scores plus faibles.

Classe	Precision	Recall	F1-Score
A	73.554	92,708	82,028
C	100.000	20.000	33.333
N	80.000	77.778	78.873

Table 11: Évaluation des performances de classification par classe de signal pour les signaux bruts et les indicateurs spéciaux.

Classe	Precision	Recall	F1-Score
A	96.806	94,792	95.789
C	89.655	86.667	88.136
N	89.744	97.222	93.333

Table 12: Évaluation des performances de classification par classe de signal pour les temps d'activation et les indicateurs spéciaux.

3.7 Résumé des performances du RandomForest

Ayant été établi précédemment la supériorité du RandomForest sur les autres classifieurs on réalise une synthèse de ces performances au tableau 13. En s'appuyant sur ce tableau on montre que les entrées donnant les meilleurs résultat sont dans cet ordre les features et les temps d'activations. Avec l'exception du tri nsr1 pour lequel les temps d'activation donnent le meilleur résultat. Le ratio d'erreur de 7.8% est très faible (l'un des seuls à passer en dessous de la barre des 10%). Il faut cependant nuancer légèrement ce résultat le score de rappel des signaux de type CHF n'étant que de 57% . Cette valeur peut être du à un défaut de nos données qui est la disparité des classes, pour rappel on dispose de 96 signaux de type ARR, 36 de types NSR et seulement 30 de type CHF. Cette disparité s'accroît d'autant plus dans le cadre du tri nsr1 où signaux ARR et NSR sont réunis sous le label NSR. Ce qui implique une faible

représentation des signaux CHF dans le jeu de test, une ou deux vont donc impacté bien plus fortement les score de CHF ou de NSR. Un autre résultat remarquable est celui donné par les features avec le tri chf puisque l'on obtient un taux de 4.45% soit le plus faible de tous avec en prime des scores tous élevés. Par ailleurs cela indique aussi que l'information semble concentrée au niveau des temps d'activation pour les signaux CHF mais pas pour les autres types. Enfin les indicateurs spéciaux z qui correspondent aux indicateurs spéciaux calculés sur les temps d'activation donnent un excellent taux d'erreur de 6%. À fortiori puisque ce taux correspond à un tri classic et s'accompagne de très bons scores. (les résultats étant satisfaisant en eux même on a pas appliqué cette méthode aux autres tris). Ce résultat est d'autant plus remarquable qu'il est cohérent avec les résultats des autres entrées. Effectivement, cette méthode combine l'utilisation des temps d'activations et de l'extraction de caractéristiques, même si ne sont pas les même caractéristiques que pour les features elles remplissent le même rôle de donner des informations à la fois ponctuelles et générales. On peut voir au travers des F1-scores de que les temps d'activations sont en général plus efficaces pour déterminer les signaux de type CHF tandis que les features performement mieux sur les signaux ARR et NSR.

Entrée	Classe	Taux d'erreur (%)	Precision (%)	Recall (%)	F1-Score (%)
— MÉTHODE DE TRI : CLASSIC —					
Raw	A	28,09	68,07	98,28	80,35
	C		14,00	2,37	4,05
	N		95,50	60,93	72,85
Activation time	A	17,33	78,85	97,06	86,85
	C		98,92	58,97	71,96
	N		91,38	65,29	74,84
Peak	A	37,91	69,76	70,64	69,86
	C		33,84	21,06	24,98
	N		55,96	72,91	62,66
Features	A	15,91	82,97	96,20	87,17
	C		76,43	54,41	61,01
	N		96,08	87,21	90,72
Reconstruct	A	28,09	68,06	98,64	80,46
	C		20,50	4,01	6,56
	N		96,25	58,30	70,30
Indicateurs Spéciaux z	A	6,00	83,67	93,79	88,44
	C		95,49	75,15	84,11
	N		88,14	78,40	83,00
MÉTHODE DE TRI : CHF					
Activation time	C	12,76	85,95	99,84	92,33
	N		99,22	46,32	61,37
Peak)	C	19,45	90,97	83,03	86,58
	N		58,19	72,41	63,54
Features	C	4,45	95,64	98,81	97,16
	N		96,04	85,00	89,60
Reconstruct)	C	15,67	83,70	99,04	90,66
	N		91,65	36,93	50,32
— MÉTHODE DE TRI : NSR1 —					
Activation time	C	7,88	99,75	57,01	71,03
	N		91,34	99,96	95,42
Peak	C	21,67	31,61	17,18	20,93
	N		83,33	91,96	87,34
Features	C	11,79	74,19	54,90	60,80
	N		90,75	95,56	93,01
Reconstruct	C	16,73	45,50	9,24	15,07

Entrée	Classe	Taux d'erreur (%)	Precision (%)	Recall (%)	F1-Score (%)
	N		83,26	99,63	90,70
— MÉTHODE DE TRI : NSR2 —					
Activation time	A		76,65	84,65	80,07
	N	24,45	75,73	63,01	67,75
Peak	A		71,64	70,86	70,88
	N	33,61	60,24	60,12	59,56
Features	A		85,64	91,24	88,06
	N	14,39	87,51	77,79	81,60
Reconstruct	A		72,76	83,36	77,50
	N	28,00	72,16	56,16	62,41

Table 13: Synthèse des performances du Random Forest.

Conclusion

Cette étude a permis de comparer différentes approches d'extraction d'informations et de classifications de signaux électrocardiogrammes. L'objectif principal était d'étudier l'intérêt du Convolutional Dictionary Learning (CDL) appliqué à ce problème. Le CDL appliqué à un signal le décompose en un faible nombre de briques élémentaires, des atomes, et les temps d'activation correspondant. Ici on utilise un CDL qui détecte trois atomes par type de signal. Nous avons confronté cette méthode à d'autres représentation des signaux. En utilisant, les données brutes, les pics des signaux, un ensemble de caractéristiques de base des signaux et des indicateurs spéciaux reposant sur la transformée en ondelettes et l'auto régression.

Nous avons appliqué ces méthode à un jeu de données de signaux ECGs divisés en trois catégories : ARR (arythmie), CHF (insuffisance cardiaques) et NSR (sain). Afin de classer les signaux nous avons appliqué à leurs différentes représentations un ensembles de classifieurs. Nous confrontons représentation des signaux et classifieurs en comparant taux d'erreur de prédiction, scores de précision, de rappel et F1-score calculés pour chacun des types de signaux. En comparant les résultats obtenus nous avons pu faire les observations suivantes. Tenter de classer les signaux en trois groupes distincts d'emblée est très peu productif et conduit à de fortes erreurs. Il vaut mieux adopter une approche où on croise des classifications avec le système un contre tous pour lequel nous réunissons deux types de signaux que nous opposons au troisième. Le classifieur donnant les meilleurs résultat est le RandomForest de scikit-learn. Comme nous pouvions nous y attendre passer les signaux bruts en entrée des classifieurs donne de très mauvais résultat même chose pour les pics. Les caractéristiques de base quant à elle permettent d'obtenir les meilleurs résultats la plupart du temps. Par exemple nous avons un résultat remarquable (en dessous des 10% de taux d'erreur) lorsque nous opposons signaux ARR et CHF aux signaux NSR. Les temps d'activations associés au CDL permettent parfois d'obtenir de meilleurs résultat notamment pour la configuration où nous opposons signaux ARR et NSR aux signaux CHF (nous passons en dessous des 10% de taux d'erreur). Nous pouvons alors supposer que pour les signaux CHF l'information est contenue dans les temps d'activation et pour les signaux ARR et NSR dans celle ci se retrouve plutôt dans les caractéristiques. La complémentarité de ces représentations du signal nous amène à calculer les indicateurs spéciaux directement sur les temps d'activation au lieu de le faire sur les signaux bruts (ce qui donne de mauvais résultat). Nous y appliquons le classifieur RandomForest et nous acquérons les meilleurs résultats de l'étude combinant un taux d'erreur très faible (largement sous la barre des 10%) pour une classification sur les trois types de signaux directement et des scores élevé pour chacun des types.

Ces résultats ouvrent la voie à des diagnostics automatisés plus fiables, bien que des tests à plus grande échelle, nous disposions probablement de trop peu de signaux CHF par exemple

(30 uniquement), ainsi que l’exploration approfondie de méthodes de conditionnement, comme les ondelettes, restent des perspectives prometteuses pour enrichir ce protocole.

References

- [1] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake VanderPlas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, pages 108–122, 2013.
- [2] Mainak Jas, Thomas Dupré La Tour, Umut Şimşekli, and Alexandre Gramfort. Learning the morphology of brain signals using alpha-stable convolutional sparse coding. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1099–1108, 2017.
- [3] Lakshmi Kuruguntla, Mamatha Bandaru, Dokku Tejaswi, Anup Kumar Mandpura, Sravan Kumar Sikhakoli, and Vineela Chandra Dodda. Geophysical data denoising using dictionary learning method with ramanujan sums for oil and minerals exploration. *Artificial Intelligence in Geosciences*, 6(2):100168, 2025.
- [4] Jérôme MOLINARO. Analyse rapide et simplifiée de l’électrocardiogramme, 2022.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [6] The MathWorks, Inc. Ecg classification using wavelet features, 2026.
- [7] The MathWorks, Inc. Introduction aux familles d’ondelettes, 2026.
- [8] Tom Dupré La Tour, Thomas Moreau, Mainak Jas, and Alexandre Gramfort. Multivariate convolutional sparse coding for electromagnetic brain signals, 2018.
- [9] Qibin Zhao and Liqing Zhang. Ecg feature extraction and classification using wavelet transform and support vector machines, 11 2005.

Annexe

Ici nous déposons le code python complet ayant été réalisé pour cette étude, téléchargeable et exploitable à la volonté du lecteur. Ainsi que les données ECGS utilisés.

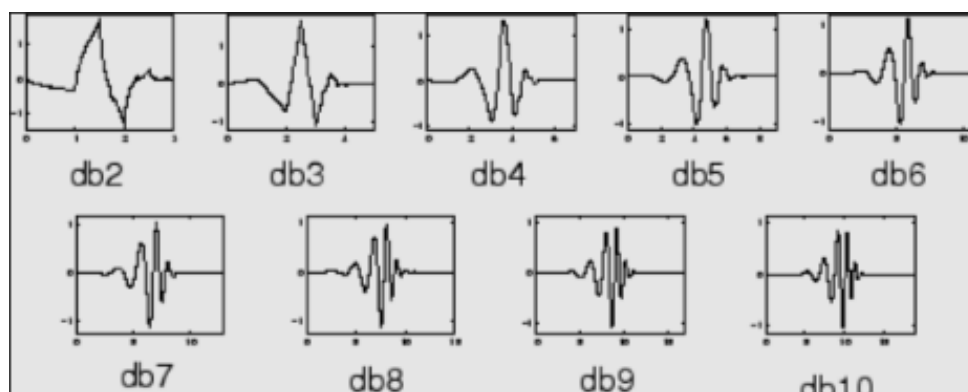


Figure 9: Ondelettes de Debauchies^[7]